

What is a container?

Imagine you're developing a killer web app that has three main components - a React frontend, a Python API, and a PostgreSQL database. If you wanted to work on this project, you'd have to install Node, Python, and PostgreSQL.

How do you make sure you have the same versions as the other developers on your team? Or your CI/CD system? Or what's used in production?

How do you ensure the version of Python (or Node or the database) your app needs isn't affected by what's already on your machine? How do you manage potential conflicts?

Enter containers!

What is a container? Simply put, containers are isolated processes for each of your app's components. Each component - the frontend React app, the Python API engine, and the database - runs in its own isolated environment, completely isolated from everything else on your machine.

Here's what makes them awesome. Containers are:

- **Self-contained.** Each container has everything it needs to function with no reliance on any pre-installed dependencies on the host machine.
- **Isolated.** Since containers run in isolation, they have minimal influence on the host and other containers, increasing the security of your applications.
- Independent. Each container is independently managed. Deleting one container won't affect any others.
- **Portable.** Containers can run anywhere! *The container that runs on your development machine will work the same way in a data center or anywhere in the cloud!*

Containers versus virtual machines (VMs)

Without getting too deep, a VM is an entire operating system with its own kernel, hardware drivers, programs, and applications. Spinning up a VM only to isolate a single application is a lot of overhead.

A kernel is the essential, core component of an operating system (OS) that acts as the primary bridge between software applications and the physical hardware

A container is simply an **isolated process** with all of the files it needs to run. If you run multiple containers, they all share the same kernel, allowing you to run more applications on less infrastructure.

Try it out

In this hands-on, you will see how to run a Docker container using the Docker Desktop GUI.

Using the GUI Using the CLI

Follow the instructions to run a container using the CLI:

1. Open your CLI terminal and start a container by using the `docker run` command:

```
$ docker run -d -p 8080:80 docker/welcome-to-docker
```

The output from this command is the full container ID.

Congratulations! You just fired up your first container! 🎉

View your running containers

You can verify if the container is up and running by using the `docker ps` command:

```
docker ps
```

You will see output like the following:

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS
a1f7a4bb3a27	docker/welcome-to-docker	"/docker-entrypoint..."	11 seconds ago	Up

This container runs a web server that displays a simple website. When working with more complex projects, you'll run different parts in different containers. For example, a different container for the frontend, backend, and database.



Tip

The `docker ps` command will show you *only* running containers. To view stopped containers, add the `-a` flag to list all containers: `docker ps -a`

Access the frontend

When you launched the container, you exposed one of the container's ports onto your machine. Think of this as creating configuration to let you to connect through the isolated environment of the container.

For this container, the frontend is accessible on port `8080`. To open the website, select the link in the **Port(s)** column of your container or visit `http://localhost:8080` in your browser.

Stop your container

The `docker/welcome-to-docker` container continues to run until you stop it. You can stop a container using the `docker stop` command.

1. Run `docker ps` to get the ID of the container
2. Provide the container ID or name to the `docker stop` command:

```
docker stop <the-container-id>
```

Tip

When referencing containers by ID, you don't need to provide the full ID. You only need to provide enough of the ID to make it unique. As an example, the previous container could be stopped by running the following command:

```
docker stop a1f
```